

SALESTORM — Architecture Decision Record (ADR)

ADR-001: Adopt a Service-Oriented Architecture

Status: Accepted

Date: 05 October 2026

Context:

SALESTORM must handle a flash sale where 10,000 customers may simultaneously attempt to purchase only 100 available units.

A single monolithic application would make independent scaling and fault isolation more difficult.

Decision:

Use a service-oriented architecture with clearly separated responsibilities:

API Gateway



Product / Cart / Checkout



Inventory & Reservation



Payment



Order



Shipment / Notification

Benefits:

- * Independent horizontal scaling
- * Clear service ownership
- * Better fault isolation

- * Easier maintenance
- * Independent evolution of services

Trade-off:

Distributed systems introduce additional network calls, monitoring requirements and operational complexity.

ADR-002: Use SQL as the Inventory Source of Truth

Status: Accepted

Context:

Inventory is the most consistency-sensitive part of SALESTORM.

The system must guarantee: $\text{Successful Sales} \leq \text{Available Stock}$

Alternatives Considered

Option	Advantages	Disadvantages
-----	-----	-----

SQL	Strong transactions and constraints	Write contention
NoSQL	High horizontal scalability	More complex consistency model
Redis	Extremely fast	Not suitable as authoritative transactional inventory

Decision:

Use a relational SQL database as the authoritative inventory store.

Reason:

SQL provides:

- * ACID transactions
- * Atomic updates
- * Constraints

- * Strong consistency

- * Reliable transaction history

Trade-off:

The inventory database can become a contention point for extremely hot products, so admission control and controlled scaling strategies are required.

ADR-003: Use Atomic Conditional Update for Inventory

Status: Accepted

Context:

10,000 customers can request the same product simultaneously.

The system must prevent this:

Available = 1

Customer A → succeeds

Customer B → succeeds

Result = -1 ❌

Decision:

Use an atomic conditional update:

UPDATE inventory

SET available_quantity = available_quantity - 1

WHERE product_id = ?

AND available_quantity >= 1;

The reservation succeeds only when the database update affects one row.

Result:

Available = 1

Request A → UPDATE succeeds → Available = 0

Request B → UPDATE fails → OUT OF STOCK

Trade-off:

High contention can create database-level contention, but correctness is prioritized over maximum write throughput.

ADR-004: Use Idempotency Keys

Status: Accepted

Context

Customers may:

- * Double-click Buy Now
- * Retry after a timeout
- * Send duplicate payment requests
- * Experience client/network retries

Decision:

Require an idempotency key for critical operations.

Example:

Idempotency-Key: BUY-12345

A unique database constraint prevents the same operation from being processed twice.

Result:

First request



Reservation created

Duplicate request



Existing result returned

Trade-off:

The system must store and manage idempotency records, increasing storage and implementation complexity.

ADR-005: Use Reservation with Expiry

Status: Accepted

Context:

Customers may reserve inventory but never complete payment.

Without expiry:

100 units



Reservations consume stock



Customers abandon checkout



Stock remains blocked **✗**

Decision:

Create temporary reservations with an expiry time.

AVAILABLE



RESERVED



PAYMENT



CONFIRMED

If payment fails or the reservation expires:

RESERVED



RELEASED



AVAILABLE

Trade-off:

A background expiry mechanism is required to detect and release expired reservations.

ADR-006: Use Transactional Outbox for Critical Events

Status: Accepted

Context:

Consider:

Payment succeeds



Order Service fails

If the payment result is not reliably recorded for later processing, the order may never be created.

Decision:

Use the Transactional Outbox Pattern.

Database Transaction



| — Payment = SUCCESS



| — Outbox Event = PaymentSucceeded

A worker publishes the event to the message broker.

Result:

Even if Order Service is temporarily unavailable:

Payment Success



Outbox



Message Broker



Retry



Order Created

Trade-off:

Requires an additional outbox table and background publisher.

ADR-007: Use Asynchronous Messaging for Downstream Work

Status: Accepted

Context:

Notification, shipment and other downstream operations should not unnecessarily block the purchase workflow.

Decision:

Use synchronous communication for critical operations and asynchronous messaging for downstream work.

Synchronous:

Checkout



Inventory



Payment

Asynchronous:

Order Confirmed



Message Broker

└──→ Shipment

└──→ Notification

└──→ Analytics

Benefits:

- * Lower customer-facing latency
- * Better fault isolation
- * Independent scaling
- * Queue-based buffering

Trade-off:

Asynchronous systems introduce eventual consistency and require retry/DLQ handling.

ADR-008: Use Circuit Breaker for External Payment Gateway

Status: Accepted

Context:

The payment gateway is an external dependency and may become slow or unavailable.

Repeated requests to an unhealthy gateway could cause cascading failures.

Decision:

Use a Circuit Breaker.

CLOSED

↓
Repeated failures

↓
OPEN

↓
Recovery test

↓
HALF-OPEN

↓
Success

↓
CLOSED

Benefits:

- * Prevents cascading failures
- * Reduces unnecessary requests
- * Allows the system to recover gracefully

Trade-off:

The application must handle the circuit-open state and payment reconciliation.

ADR-009: Use Cache for Read-Heavy Product Data

Status: Accepted

Context:

Product information receives many more reads than writes during a flash sale.

Sending every product request to the database increases load unnecessarily.

Decision:

Use a distributed cache for:

- * Product details
- * Categories
- * Sale metadata
- * Frequently accessed read-only information

Request



Cache

| — HIT → Response



| — MISS



Database

Important Constraint:

Inventory availability is not treated as ordinary cache data. The transactional database remains the inventory authority.

Trade-off:

Cached information may become temporarily stale and therefore requires TTL/invalidation strategies.

ADR-010: Use API Gateway Admission Control

Status: Accepted

Context:

The flash sale can produce a sudden request spike. The architecture must protect backend services from uncontrolled traffic.

Decision:

Implement:

- * Rate limiting
- * Token bucket admission control
- * Request validation
- * Authentication
- * Routing
- * Traffic throttling

10,000 Requests



API Gateway



Admission Control



Backend

Benefits:

- * Protects backend services
- * Controls traffic bursts
- * Improves system stability

Trade-off:

Some legitimate requests may be rejected or delayed during extreme overload.

ADR-011: Prefer Horizontal Scaling for Stateless Services

Status: Accepted

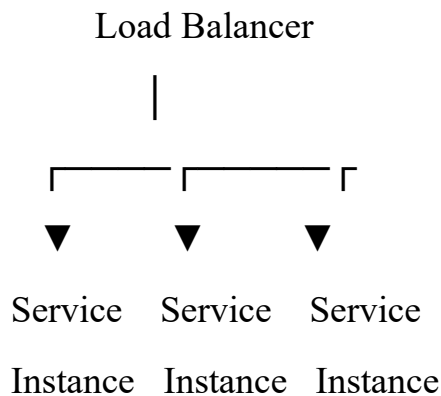
Context :

Traffic can increase dramatically during the sale.

A single application instance cannot reliably handle unpredictable traffic growth.

Decision:

Keep application services stateless and scale horizontally.



Benefits:

- * Increased throughput
- * Better availability
- * Easy instance replacement
- * Supports autoscaling

Trade-off:

Requires shared infrastructure for state such as databases, caches and message brokers.

ADR-012: Prioritize Consistency for Inventory

Status: Accepted

Context:

SALESTORM is a limited-stock flash sale.

Selling an item that does not exist is more damaging than temporarily rejecting a purchase.

Decision:

Prioritize strong consistency for the inventory reservation boundary.

The system may reject a request when inventory cannot be safely verified.

It must never accept a reservation that violates inventory integrity.

Core invariant: `available_quantity >= 0` and `OVERSOLD_COUNT = 0`

Trade-off:

During some failure conditions, availability may temporarily decrease rather than allowing potentially incorrect purchases.

ADR Summary Matrix

ADR	Decision	Main Reason	
-----	-----	-----	
ADR-001	Service-oriented architecture	Scalability & separation	
ADR-002	SQL inventory authority	Strong consistency	
ADR-003	Atomic conditional update	Prevent overselling	
ADR-004	Idempotency keys	Prevent duplicates	
ADR-005	Expiring reservations	Release abandoned stock	
ADR-006	Transactional outbox	Reliable event delivery	
ADR-007	Async messaging	Resilience & scalability	
ADR-008	Circuit breaker	External failure isolation	
ADR-009	Cache product data	Reduce database load	
ADR-010	API admission control	Protect backend	
ADR-011	Horizontal scaling	Handle traffic growth	
ADR-012	Consistency-first inventory	Guarantee correctness	

Key Architecture Trade-offs

Decision	We Gain	We Sacrifice	
-----	-----	-----	

SQL over NoSQL for inventory	Strong consistency	Some write scalability	
Atomic update	Zero overselling	Database contention	
Sync inventory/payment	Immediate correctness	Higher latency	
Async downstream events	Resilience	Eventual consistency	
Transactional outbox	Reliable events	Additional infrastructure	
Cache	Lower read load	Possible stale data	
Rate limiting	System protection	Some requests rejected	
Circuit breaker	Fault isolation	Temporary unavailability	
Microservices	Independent scaling	Distributed-system complexity	

Final ADR Statement

SALESTORM's architecture prioritizes correctness at the inventory boundary while using horizontal scaling, caching, admission control and asynchronous processing to handle high traffic. Critical operations such as inventory reservation and payment are protected through atomic transactions and idempotency, while distributed failures are handled using transactional outbox, retries, circuit breakers and reconciliation. These decisions intentionally trade some availability and implementation simplicity for strong inventory integrity and reliable order processing.